IOT NA PRÁTICA

ESP32 + MicroPython

Aula 1 — Sensor, Wi-Fi e HTTP

Especialização · Primeiro contato com hardware

O que é IoT, no projeto de hoje



É exatamente esse ciclo que vamos construir, do zero, nas próximas duas horas.

Conhecendo o ESP32

- Microcontrolador com **Wi-Fi e Bluetooth** embutidos
- Alimentado por **cabo USB** — nada de fonte externa hoje
- LED azul já soldado na placa, no **GPIO 2**
- Sensor de temperatura **interno ao chip** — sem fiação extra

Hoje não precisamos de

protoboard, resistores ou solda

só o cabo USB e o notebook

Preparando o ambiente: instalar o Thonny

1

Baixar e instalar o Thonny

thonny.org — já vem com suporte a MicroPython embutido, sem plugin extra

2

Conectar o ESP32 via USB

Usar um cabo USB de dados — muitos cabos são só de carga e não funcionam aqui

Se a porta não aparecer no sistema: falta instalar o driver USB-serial (CP210x ou CH340, dependendo da placa)

Gravando o MicroPython no ESP32

1. Menu **Ferramentas → Opções → Interpretador**
2. Selecionar **"MicroPython (ESP32)"** na lista de interpretadores
3. Clicar em **"Install or update MicroPython (esptool)"**
4. Escolher a **porta COM** e a variante genérica do ESP32, depois clicar em **Install**
5. Aguardar cerca de 1 minuto — pronto quando o console mostra o prompt **>>>**

⚠

Se travar

Fechar e reabrir o Thonny costuma resolver quando a porta "some" depois de um erro

Firmware corrompido não é problema: repetir o processo sobrescreve normalmente

BLOCO 1

Hello World do hardware: piscar o LED

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)

while True:
    led.value(not led.value())
    time.sleep(0.5)
```

BLOCO 2

Lendo o sensor de temperatura

```
import esp32
import time

while True:
    temp_c = (esp32.raw_temperature() - 32) / 1.8
    print("Temperatura:", temp_c, "°C")
    time.sleep(2)
```

Sensor interno do chip: zero fiação extra — mas superestima a temperatura ambiente porque aquece com o próprio processador.

BLOCO 3

Conectando ao Wi-Fi

```
import network
import time

SSID = "NOME_DA_REDE"
PASSWORD = "SENHA_DA_REDE"

def conectar_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        wlan.connect(SSID, PASSWORD)
        while not wlan.isconnected():
            time.sleep(0.5)
    print("Conectado:", wlan.ifconfig())

conectar_wifi()
```

BLOCO 4

Publicando a leitura via HTTP

```
import urequests

URL = "https://webhook.site/SEU-ID-AQUI"
LIMIAR_TEMP = 30

while True:
    temp_c = (esp32.raw_temperature() - 32) / 1.8
    alerta = temp_c > LIMIAR_TEMP
    led.value(alerta)

    try:
        resposta = urequests.post(URL, json={"temperatura": temp_c, "alerta": alerta})
        print("Enviado, status:", resposta.status_code)
        resposta.close()
    except Exception as e:
        print("Erro ao enviar:", e)

    time.sleep(5)
```

webhook.site: endpoint gerado na hora, sem cadastro — a turma vê a requisição chegando ao vivo no navegador.

O que acabamos de construir



Do firmware ao dado na nuvem: o mesmo ciclo por trás de qualquer solução de IoT industrial.

Próxima aula: o mesmo projeto, com MQTT

Hoje (HTTP)

Requisição pontual, um envio por vez

Próxima aula (MQTT)

Conexão persistente, publish/subscribe, mais eficiente em escala